

From Unstructured Text to Knowledge Graph: An Automated Pipeline for Ontology Discovery, Entity Resolution, and Graph-RAG via LLM-FalkorDB Integration

Alessandro Bevilacqua¹

¹ Department of Computer Science, University of Bari Aldo Moro, Via Orabona 4, 70125 Bari, Italy

Abstract

Research into Retrieval-Augmented Generation (RAG) has surged in both industry and academia, primarily due to the tendency of Large Language Models (LLMs) to hallucinate in technical and specialized domains. **Graph-RAG** has emerged as a compelling solution, leveraging Knowledge Graphs (KGs) to ground LLM outputs. Unlike unstructured data, KGs provide a structured logical framework that enhances reasoning and enables Explainable AI (XAI) by providing traceable justifications.

In this study, we propose a novel framework to automate Graph-RAG construction from unstructured text using LLMs and **FalkorDB**. Our pipeline first partitions documents by category, after which an LLM constructs a validated ontology for each category to guide entity and relation extraction. To ensure graph integrity, we perform **Entity Resolution** by converting entities to text, clustering their embeddings via HDBSCAN, and employing an LLM to merge duplicates (if any) within each cluster. Finally, the resulting KGs (one for each category) are queried through an LLM-orchestrated approach, with performance evaluated using an **LLM-as-a-judge** methodology. As a case study, we developed a specialized system based on GPT-5-mini, optimized to answer queries related to **EmPULIA**¹ and the broader **Italian Public Procurement Code** (“Codice degli Appalti”). This case study is particularly compelling as the source material consists exclusively of highly technical, unstructured text documents, without access to structured datasets or alternative data formats.

Experimental results demonstrate that our pipeline significantly enhances LLM performance, increasing GPT-5-mini accuracy to **63%**. This outperforms both the standalone model accuracy (**29%**) and its web-enhanced configuration (**42%**).

Keywords

Knowledge Graph, Graph-RAG, LLM, Entity Resolution, HDBSCAN, FalkorDB, CEUR-WS

1. Introduction and Motivations

The Graph-RAG approach uses knowledge graphs to provide substantial improvements to LLMs in question-and-answer performance when reasoning about complex information. RAG techniques have shown promise in enabling LLMs to reason about private datasets, i.e. data that the LLM is not trained on and has never seen before, such as business documents. However, the construction of Knowledge Graphs is a non-trivial task, particularly in

¹ EmPULIA is the official regional purchasing center and e-procurement platform for the Apulia Region in Italy, managed by InnovaPuglia S.p.A.



complex domains, and requires attention to Entity Resolution³, data consistency and scalability. The traditional methodology for KG construction, particularly when dealing with unstructured data, typically demands extensive manual effort. This human-centric process is not only time-consuming but also difficult to scale, creating a significant bottleneck for a wide range of applications. Recently, the paradigm has shifted toward LLM-driven KG construction, and many algorithms have been proposed. While current methods show promise in general-purpose applications, their efficacy diminishes in highly specialized or technical fields, such as biology and law. Consequently, the academic community is actively exploring more robust solutions. In this study, we propose an automated pipeline for LLM-driven KG construction, designed to significantly enhance LLM performance through a Graph-RAG-based methodology. The EmpULIA case study serves as an ideal testing ground, as it allows us to evaluate our framework within a challenging and highly specialized domain.

2. Related Work

In the survey paper [1] 'Can Knowledge Graphs Reduce Hallucinations in LLMs?', the authors propose a comprehensive taxonomy for KG-Augmented Large Language Models, categorizing them into three primary types:

- **Knowledge-aware inference:** This method integrates KGs to provide structured symbolic knowledge to LLMs, primarily by incorporating KGs at the input level to enhance contextual understanding.
- **Knowledge-aware training:** This approach leverages KGs to optimize the model's learning process, either by enhancing the quality of training data during pre-training or by fine-tuning the model to adapt to specific tasks or domains.
- **Knowledge-aware validation:** This approach uses structured data as a fact-checking mechanism, providing a reference for the model to verify information.

Our pipeline is categorized under Knowledge-aware inference, a methodology that has been significantly advanced by the introduction of **Microsoft's GraphRAG** [2] framework in 2024. Unlike traditional Knowledge Graph RAG systems that rely on simple neighborhood retrieval, this framework introduces **hierarchical community detection**. This allows the system to generate high-level summaries of interconnected data clusters, enabling efficient global queries and overcoming the contextual fragmentation often found in standard vector-based retrieval.

A notable recent development is **KGGen** [3], a text-to-knowledge-graph generator that employs LLMs to extract structured information from unstructured text. Its core technical innovation is a novel entity resolution approach that actively clusters and de-duplicates

³ Entity resolution is the data management process of identifying, linking, and merging disparate records that represent the same real-world entity—such as a person, organization, or product—across different data sources.

related entities during the extraction process. Unlike standard extractors that often produce sparse or redundant nodes, KGGen focuses on reducing sparsity to create more concise and generalizable entities and relations.

3. Proposed Approach

Our automated pipeline consists of three primary phases:

1. **Knowledge Graph Construction:** GPT-5-mini is used to extract and structure data.
2. **Entity Resolution:** HDBSCAN is employed to identify clusters, and GPT-5-mini is used to resolve duplicate entities within each cluster.
3. **Inference:** An orchestrator-based approach integrates the generated KGs with GPT-5-mini to enhance question answering accuracy and mitigate hallucinations.

3.1. Knowledge Graph Construction

The dataset comprises PDF documentation concerning the EmPULIA e-procurement platform and the Italian Public Procurement Code. These documents were pre-classified by category at the source; consequently, we elected to maintain this structure by developing a distinct Knowledge Graph (KG) for each thematic area. The categories are:

1. **CodiceAppalti:** This represents the primary legislative foundation. It contains the statutory rules governing how Italian public administrations must award contracts for works, services, and supplies, defining principles such as transparency, competition, and legal thresholds.
2. **DisciplinaDiUtilizzo:** This category encompasses the contractual and regulatory framework for the EmPULIA platform itself. It outlines the legal terms, conditions for access, and the responsibilities of users (both public and private) when interacting with the digital system.
3. **FAQ:** A collection of recurring queries and official answers.
4. **GuidePraticheOE:** Technical documentation specifically tailored for businesses (suppliers). These guides explain the end-to-end workflow from the supplier's perspective.
5. **GuidePraticheSA:** Technical documentation dedicated to public buyers (PA).
6. **Normativa:** This includes secondary legislation, ministerial circulars, and ANAC (National Anti-Corruption Authority) guidelines.

To convert the PDF files into .txt files, we use the **PyPDF2's PdfReader** library and then apply a preprocessing function that performs:

- **Normalization:** It normalizes Unicode (fixing different styles of accents/apostrophes) and removes non-printable characters often found in PDF exports.

- **Spacing Cleanup:** It removes unnecessary spaces before punctuation and around hyphens (e.g., changing "word - word" to "word-word").
- **Structure Compression:** It collapses three or more consecutive empty lines into a single line break to keep the text compact.
- Additional refinements in which regular expressions are used to correct formatting issues.

Adopting the terminology established in the survey 'LLM-Empowered Knowledge Graph Construction: A Survey' [4], we implemented a **schema-based** extraction framework, first developing a tailored ontology for each KG to provide structural guidance throughout the process.

Ontology creation is delegated to GPT-5-mini; given a text, its primary objective is to generate an ontological schema using the JSON format, as it is a widely adopted data format for LLM interactions. This choice is motivated by the fact that LLMs undergo extensive pre-training on JSON-formatted data, such as web APIs and software configurations, making them highly proficient in this syntax compared to other serialization formats. JSON has been shown to significantly reduce ambiguity and compels the model to function as a data processor rather than a creative writer. This approach is ideal for ensuring consistent, structured output while minimizing the risk of syntactic hallucinations. To manage document sets that are too large for single-stage processing, we adopted the following strategy for ontology discovery:

```

Algorithm 1: Ontology extraction and hierarchical merging process
via LLM
Input : Topic list  $\mathcal{T}$ , Maximum number of characters  $M = 20,000$ 
Output: An ontology for each topic
1  $ontology\_list \leftarrow \emptyset$ 
2 foreach topic  $t \in \mathcal{T}$  do
3    $O_{pool} \leftarrow \emptyset$  // Initialize ontology container
4   foreach document  $d \in t$  do
5     if  $length(d) > M$  then
6        $chunks \leftarrow \text{split } d \text{ in text chunks of size } \leq M$ 
7     end if
8     else
9        $chunks \leftarrow \langle d \rangle$ 
10    end if
11    foreach chunk  $ch \in chunks$  do
12       $ont \leftarrow \text{LLM.Generate\_JSON\_Ontology}(ch)$ 
13       $O_{pool}.append(ont)$ 
14    end foreach
15  end foreach
16  // Hierarchical Merging Phase
17  while  $|O_{pool}| > 1$  do
18     $O_{next} \leftarrow \emptyset$ 
19    for  $i = 0$  to  $|O_{pool}| - 1$  do
20      if  $i + 1 < |O_{pool}|$  then
21         $merged \leftarrow \text{LLM.Merge}(O_{pool}[i], O_{pool}[i + 1])$ 
22         $O_{next}.append(merged)$ 
23      end if
24      else
25         $O_{next}.append(O_{pool}[i])$ 
26      end if
27    end for
28     $O_{pool} \leftarrow O_{next}$ 
29  end while
30   $ontology\_list.append(O_{pool}[0])$ 
31 end foreach
32 return  $ontology\_list$ 

```

The algorithm processes documents by category, splitting those exceeding a character number threshold into smaller chunks. An LLM generates a JSON ontology for each chunk. These ontologies are then iteratively merged in a binary-tree style until a single, unified ontology is produced for each category.

Note: A sliding window approach is employed for document splitting, where each chunk overlaps with the previous one to prevent loss of context at the boundaries.

To ensure the generation of valid JSON ontologies, GPT-5-mini is prompted with a formal JSON schema, specific formatting constraints, and a one-shot demonstration. Furthermore,

if the generated JSON is invalid—either due to syntax errors (e.g., missing brackets) or schema violations (e.g., duplicate entities or missing identity attributes)—the LLM is prompted to regenerate the ontology, guided by a structured description of the identified errors.

The next stage involves extracting data (entities and relations) guided by the previously generated ontologies. We maintain the same chunk-based processing approach to ensure scalability. The system enforces ontological consistency by validating the JSON output; any non-compliant results are returned to the LLM for iterative correction.

Algorithm 1: Data extraction through LLM

Input : Topic list $\mathcal{T}$, Maximum number of characters $M = 10,000$
Output: A JSON object for each topic, containing all the extracted entities and relations

```

1  $json\_data\_list \leftarrow \{\}$ 
2 foreach topic  $t \in \mathcal{T}$  do
3    $O_{pool} \leftarrow \{\}$  // Initialize JSON data container
4   foreach document  $d \in t$  do
5     if  $length(d) > M$  then
6        $chunks \leftarrow \text{split } d \text{ in text chunks of size } \leq M$ 
7     end if
8     else
9        $chunks \leftarrow \langle d \rangle$ 
10    end if
11    foreach chunk  $ch \in chunks$  do
12       $data \leftarrow \text{LLM\_Extract\_JSON\_Data}(ch)$ 
13       $O_{pool}.append(data)$ 
14    end foreach
15  end foreach
16   $json\_data\_topic \leftarrow \text{Aggregate\_All\_JSON}(O_{pool})$ 
17   $json\_data\_topic \leftarrow \text{LLM\_Remove\_Duplicate\_Entities}(json\_data\_topic)$ 
18   $json\_data\_topic \leftarrow \text{LLM\_Remove\_Duplicate\_Relations}(json\_data\_topic)$ 
19   $json\_data\_list.append(json\_data\_topic)$ 
20 end foreach
21 return  $json\_data\_list$ ;

```

Once the LLM extracts data from each text chunk for a specific category, all entities and relations are aggregated into a single JSON object. Subsequently, any duplicate entities and relations are identified, and the LLM is prompted to merge them.

Note: Entities are considered duplicates if they share the same label and the same key attribute value. Relations are considered duplicates if they share the same label and connect the same source and target entities.

3.2. Entity Resolution

We represent each entity as text using its labels, attributes and 1-hop relations. Finally, these text descriptions are converted into embeddings using the *text-embedding-3-small* model.

To process these high-dimensional vectors effectively, we implement a dimensionality reduction step using **UMAP** (Uniform Manifold Approximation and Projection). Given that the embeddings reside in a 1,536-dimensional space, direct clustering often suffers from the curse of dimensionality, where distance metrics become less discriminative. UMAP compresses these embeddings into a lower-dimensional manifold while specifically preserving the local structure. This ensures that entities with similar semantic descriptions and shared 1-hop relations remain proximal, effectively "concentrating" the density of related records. These reduced embeddings are then clustered via **HDBSCAN** (Hierarchical Density-Based Spatial Clustering of Applications with Noise). Unlike centroid-based methods (e.g., K-Means), HDBSCAN does not require a pre-defined number of clusters. This is vital for ER, where the total number of unique real-world entities is unknown a priori.

This architecture mirrors the **BERTopic** framework [5], treating the entity resolution task as a search for "semantic islands" within the data.

The final clusters are passed to GPT-5-mini, which serves as the ultimate arbiter. The LLM performs the final verification, deciding which entities represent the same real-world object and how to reconcile any conflicting data during the merge.

3.3. Inference

The final stage of the pipeline employs **GraphRAG-SDK** to query the generated KGs. An orchestrator agent directs the workflow by creating execution plans and delegating tasks to category-specific sub-agents. By leveraging these specialized agents to traverse individual KGs, the system ensures highly accurate, domain-specific retrieval when answering user questions.

We translated and customized the default prompts to align with the Italian Public Procurement Code domain. We specifically moved away from a conversational Q&A style toward a direct-response format. This adjustment was essential for the LLM-as-a-judge evaluation framework, which requires precise, non-discursive outputs for accurate scoring.

4. Evaluation

The evaluation framework follows a structured, automated pipeline:

- **Sampling:** We randomly select a category, a source document of that category, and a specific text chunk from that document.
- **Question Generation:** GPT-5-mini (acting as the synthetic user) generates a targeted question based specifically on the selected text chunk.
- **Inference:** The model under evaluation attempts to answer the generated question.
- **Automated Assessment:** GPT-5-mini (acting as the LLM-as-a-judge) performs a binary evaluation of the response, determining whether it is correct and grounded in the source text.

We recorded a dataset of 139 questions in CSV format to perform a comparative analysis between our GPT-5-mini Orchestrator Agent augmented with Knowledge Graphs, a baseline GPT-5-mini model using only its training data (without RAG capabilities), and the same GPT-5-mini model enhanced with web search results retrieved via Tavily⁴ and included in the prompt.

⁴ Tavily is a specialized web search engine and API designed specifically for AI agents.

5. Results

Table 1 GPT-5-mini with KG results

RESULTS	Accuracy	# correct answers	# wrong answers	# total answers
Global	0.63	88	51	139
"DisciplinaDiUtilizzo"	0.69	22	10	32
"CodiceAppalti"	0.62	29	18	47
"FAQ"	1	12	0	12
"GuidePraticheOE"	0.55	16	13	29
"GuidePraticheSA"	0.47	9	10	19

Table 2 GPT-5-mini with web results

RESULTS	Accuracy	# correct answers	# wrong answers	# total answers
Global	0.42	59	80	139
"DisciplinaDiUtilizzo"	0.47	15	17	32
"CodiceAppalti"	0.53	25	22	47
"FAQ"	0.5	6	6	12
"GuidePraticheOE"	0.24	7	22	29
"GuidePraticheSA"	0.32	6	13	19

Table 3 GPT-5-mini standalone results

RESULTS	Accuracy	# correct answers	# wrong answers	# total answers
Global	0.29	40	99	139
"DisciplinaDiUtilizzo"	0.38	12	20	32
"CodiceAppalti"	0.32	15	32	47
"FAQ"	0.5	6	6	12
"GuidePraticheOE"	0.1	3	26	29
"GuidePraticheSA"	0.21	4	15	19

6. Conclusion and limitations

Our pipeline demonstrates a significant **improvement in LLM accuracy** when addressing highly domain-specific queries. Despite these gains, the 63% accuracy achieved in the EmpULIA case study remains constrained by several factors. First, the extraction of text from PDF sources introduced persistent formatting noise, even after the regex corrections. Furthermore, the GuidePratiche documents contain numerous images, resulting in fragmented text that lacks critical visual information.

A primary challenge identified was the difficulty GPT-5-mini faced during ontology construction. We observed that the model struggled to find the **optimal level of**

abstraction, alternating between over-generalization and excessive granularity in defining entities and relations. This necessitated iterative prompt engineering to calibrate the model's output. Consequently, we anticipate that providing a pre-defined base ontology, or a fully established one where available, would significantly enhance performance

References

- [1] Agrawal, G., Kumarage, T., Alghamdi, Z., & Liu, H. (2024). Can Knowledge Graphs Reduce Hallucinations in LLMs?: A Survey. arXiv preprint arXiv:2404.14446.
- [2] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., & Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv preprint arXiv:2404.16130.
- [3] Mo, B., Yu, K., Kazdan, J., Cabezas, J., Mpala, P., Yu, L., Cundy, C., Kanatsoulis, C., & Koyejo, S. (2024). KGGen: Extracting Knowledge Graphs from Plain Text with Language Models. arXiv preprint arXiv:2410.12154.
- [4] H. Bian, "LLM-Empowered Knowledge Graph Construction: A Survey," Xidian University, Xi'an, China, Tech. Rep., 2024.
- [5] M. Grootendorst, "BERTopic: Neural topic modeling with a class-based TF-IDF procedure," arXiv preprint arXiv:2203.05794, 2022.